

Cutting False Alarms in Real-Time Cache Side-Channel Detection Using Deep Sequence Models

Thesis-1 Research Proposal | Ahmed Khan | MS Artificial Intelligence, FAST-NUCES

1. Problem

Cloud providers run many customers on one physical CPU, and those customers share the last-level cache. An attacker who rents a machine next to a victim can watch that shared cache and steal secrets, including private encryption keys, without breaking through any software boundary. The main defence today is a machine learning detector that reads hardware performance counters many times a second to decide whether an attack is running. Ghabbara and Trifa (ICIW 2025) [1] built one of the strongest such systems, using Random Forest and XGBoost with real-time mitigation, and reported roughly 96 percent accuracy. Their own conclusion names two open problems: the false alarm rate still needs work, and future research should integrate deep learning. False alarms are the more serious. A detector that keeps accusing innocent programs gets switched off, and in their design a false alarm also triggers noise injection, which slows an honest customer down. Joshi et al. (2025) [2] have since applied deep learning here, but on clean data with no system load, and they optimise for accuracy rather than false alarms. Nobody has tested whether deep learning reduces false alarms under load, which is where the problem appears.

2. Foundation and Contribution

Where the Literature Stands	What This Thesis Adds
Ghabbara and Trifa (2025) [1] detect and mitigate attacks in real time using Random Forest and XGBoost over hardware counters sampled every 50 microseconds, tested on both an idle and a fully loaded machine. They report the false alarm rate as their main unsolved problem and name deep learning as the next step. Joshi et al. (2025) [2] apply deep learning and report high accuracy, but their data is collected on an unloaded machine and their evaluation targets accuracy, not false alarms.	A deep sequence model that reads the raw counter signal in time order rather than a set of summary statistics, trained and tested inside the loaded and unloaded protocol of the base paper. A like-for-like comparison against a Random Forest and XGBoost baseline that I reproduce on the same machine and the same recordings, with false alarm rate as the headline metric rather than accuracy.

The idea behind the model is simple, and it explains why false alarms happen now:

- **An attack has a rhythm.** The attacker flushes a cache line, waits, times how long the reload takes, and repeats. That steady repeating cycle, not the sheer volume of cache activity, is what really marks an attacker.
- **Averaging throws the rhythm away.** The current method squeezes each time window into averages before the model ever sees it, so all that survives is how busy the cache was. A heavy but honest program then looks like an attacker. A sequence model reads the window in order and can still see the pattern.

This gives a prediction that can be proved wrong: if rhythm is the real signal, a model able to see it should raise far fewer false alarms while still catching the attack. If it does not, the limitation lies in the counters rather than the model, which is a useful finding either way.

3. Why This Problem, and How It Will Be Evaluated

Why this testbed	Evaluation plan
Every alarm can be checked. I control which process is the attacker, so a false alarm is counted exactly rather than estimated. This is what makes false alarm rate a trustworthy headline number here. The baseline is fully documented. The base paper publishes its counters, sampling interval, tools and load protocol, so I can rebuild it faithfully and any improvement can be credited to the model rather than to a different setup.	Setup: an Intel machine on bare metal, four hardware counters sampled every 50 microseconds through perf, following Ghabbara and Trifa (2025) [1]. Data: Mastik implementations of Flush+Reload, Prime+Probe and Flush+Flush as attacks, with MiBench and ordinary desktop workloads as benign traffic, each recorded idle and under full load over repeated runs. Models: the reproduced Random Forest and XGBoost baseline against a 1D CNN, an LSTM and a temporal

Why this testbed	Evaluation plan
	convolutional network trained on raw counter sequences. Metrics: false alarm rate first, then detection rate and F1, plus per-window inference time to confirm the model is still fast enough for real-time use.

4. Illustrative Example

A bank runs its payment service in a cloud VM. An attacker rents a cheap VM on the same physical server, so the two share the last-level cache. The attacker repeatedly flushes the cache lines holding the OpenSSL signing code and times each reload. A fast reload means the bank just used that line, and from the pattern of which lines were used the attacker rebuilds the private TLS key. Nothing is logged, because no firewall is crossed and no bug is exploited. Now the defence side. On that same server a video service is compressing files, which pounds the cache in a heavy but honest way. A detector that mainly counts how busy the cache is will flag it, and the mitigation will slow a paying customer who did nothing wrong. After a few such incidents the provider turns the detector off, and the bank is left unprotected. Bringing that false alarm rate down, without missing the real attacker, is the aim of this thesis.

5. Significance

Practically, a detector is only kept switched on if its false alarms are rare, so reducing them is what moves this work from a laboratory result toward something a cloud provider would deploy. Theoretically, it tests a specific claim about the signal: that the timing pattern of cache activity, not its volume, separates an attacker from a busy neighbour. The deliverables are a labelled dataset recorded idle and under load, a reproduced traditional baseline, three deep sequence models, and a comparison reporting false alarm rate alongside detection rate and inference speed. A pilot study last semester on simulated cache traces gives me the pipeline to begin immediately.

6. References

- [1] Y. Ghabbara and Z. Trifa, "A real-time cache side channel attack detection and mitigation framework based on machine learning," in *Proc. 20th Int. Conf. Internet and Web Applications and Services (ICIW)*, IARIA, 2025.
- [2] Joshi *et al.*, "Hybrid deep learning model for multiple cache side channel attacks detection: A comparative analysis," *arXiv preprint arXiv:2501.17123*, 2025.
- [3] Tong *et al.*, "Cache side-channel attacks detection for AES encryption based on machine learning," in *Proc. Int. Conf. Intelligent Computing (ICIC)*, Lecture Notes in Computer Science, vol. 14875, Springer, 2024.
- [4] D. Kosasih, F. Feng, C. Chuengsatiansup, Y. Yarom, and Z. Zhu, "Can we really detect cache side-channel attacks by monitoring performance counters?," in *Proc. ACM Asia Conf. Computer and Communications Security (AsiaCCS)*, 2024.